JDK 1.0 Story :

* In JDK 1.0, Java wanted to **store the objects** with dynamically growable nature so they introduced a predefined class called **Vector** available in java.util package from JDK 1.0V

Example :

```
Vector<Instagram> v = new Vector<>();
v.add(new Instagram(1, "Scott"));
v.add(new Instagram(2, "Scott"));
v.add(new Instagram(3, "Scott"));
v.add(new Instagram(4, "Scott"));
```

* In the same version, email applications were becoming very popular, Java wanted to store these email applications using java language, first choice was Vector.

Vector user = new Vector();   Vector pwd = new Vector();
~~user.add("r@gmail.com");~~   pwd.add("r123456");
user.add("p@gmail.com");   pwd.add("pp%^&**");
user.add("q@gmail.com");   pwd.add("q7788");

user.remove("r@gmail.com");

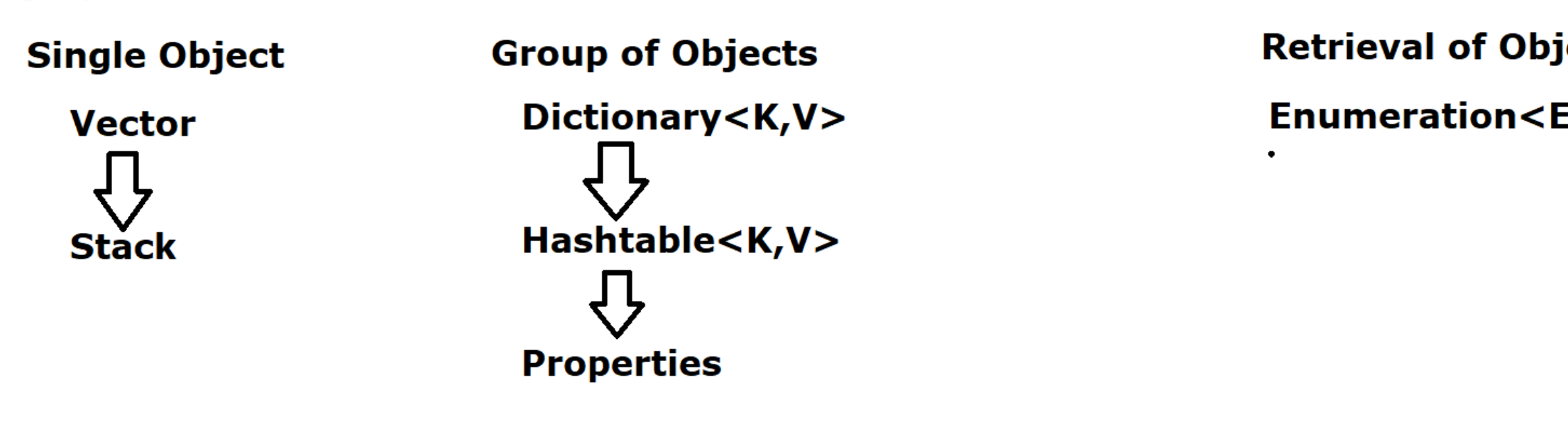
Error Log : There is no mapping between these two Vector application

* In order to hold user and password i.e **key and value pair, they introduced another class called Dictionary<K,V> which is an abstract class and to provide the implementation of abstract methods of Dictionary<K,V> class they introduced another class called Hashtable<K,V> available in java.util pacakge from JDK 1.0V**

IQ :

What are the legacy classes & interfaces in Java ?

* All the collection classes & interfcaes, which are introduced before Collection interface (JDK 1.2) are called legacy classes.



JDK 1.2V story :

* Java decided to re-built OR re-structure with Layered Architecture approach. (Reverse Engineering)

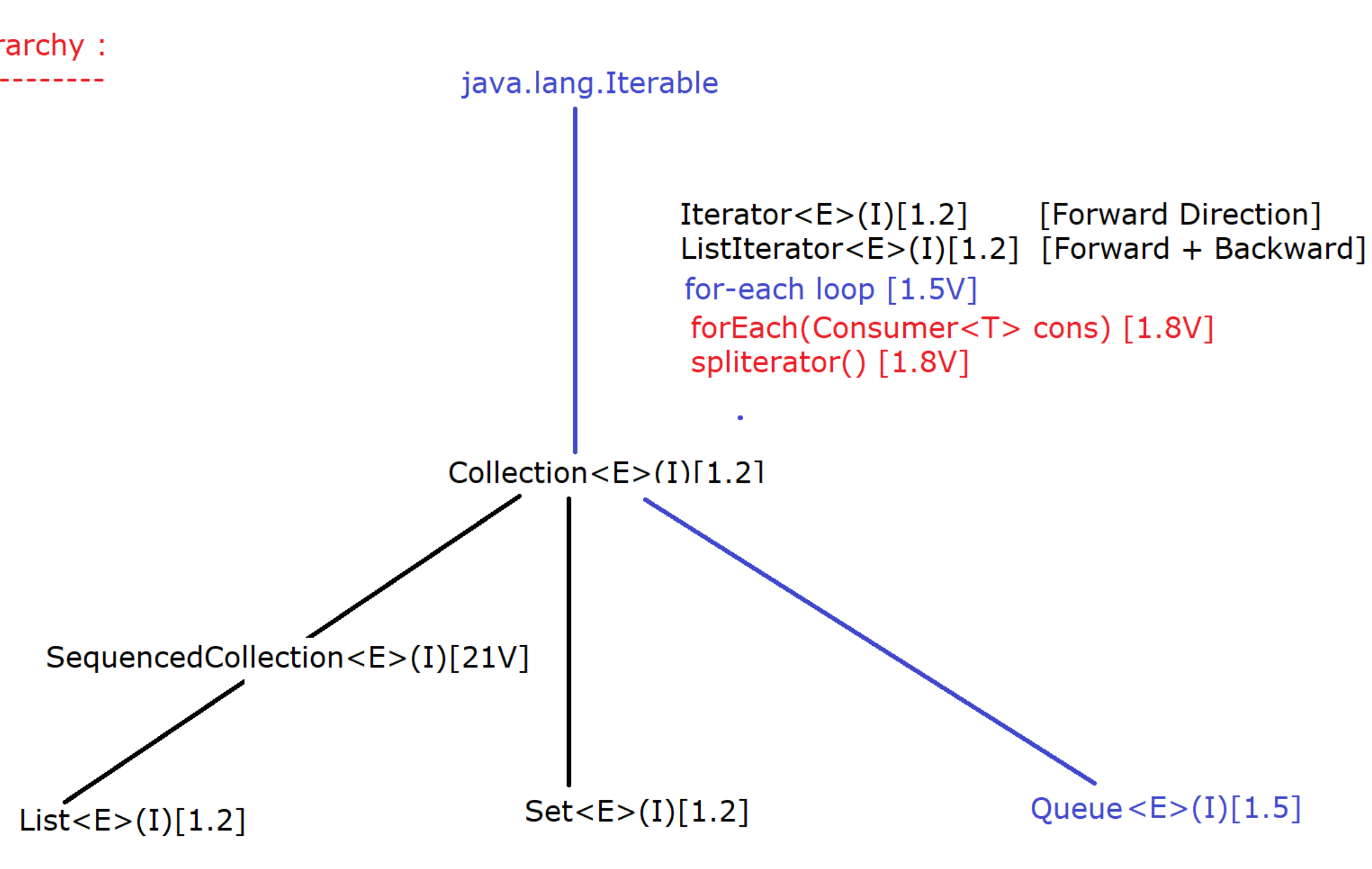
Layer 1 : Only interfaces [Full Abstarction]

Layer 2 : Only Abstract classes [Partial Abstraction]

Layer 3 : Concrete classes [No Abstraction]

* In JDK 1.2V, Collection & Map interafaces came into picture.

Collection Hierarchy :


java.lang.Iterable
Iterator<E>(I)[1.2]   [Forward Direction]
ListIterator<E>(I)[1.2] [Forward + Backward]
for-each loop [1.5V]
forEach(Consumer<T> cons) [1.8V]
spliterator() [1.8V]
Collection<E>(I)[1.2]
SequencedCollection<E>(I)[21V]
List<E>(I)[1.2]
Set<E>(I)[1.2]
Queue<E>(I)[1.5]

Iterable<T> :

* It is a predefined interface available in java.lang package from JDK 1.5V. Any object which implemets from java.lang.Iterable<T> will become iterable object so, we can apply for-each loop like array & collection

Collection<E>

* It is a predefined interface available in java.util package from JDK 1.2V. It represents the root hierarchy for all the collection classes & interfaces.

List<E>

* It is a predefined interface available in java.util package from JDK 1.2V. It can accept duplicate objects.

Set<E>

* It is a predefined interface available in java.util package from JDK 1.2V. It does not accept duplicate objects.

Queue<E>

* It is a predefined interface available in java.util package from JDK 1.5V. It works on FIFO (First In First Out).

Iterator<E>

* It is a predefined interface available in java.util package from JDK 1.2V. It is used to read the collection object in forward direction only.

ListIterator<E>

* It is a predefined interface available in java.util package from JDK 1.2V. It is used to read the collection object in both the direction i.e **forward and backward**.

SequencedCollection<E>

* It is a predefined interface available in java.util package from JDK 21V. It is used to represent the collections which follows **sequence OR some specific order.**

Collection**s :**

* It is a predefined utility class available in java.util package from JDK 1.2V. It provides various static methods to work with collection object.

Methods of Collection interface :

a) public boolean add(E element) :- It is used to add an item/element in the in the collection.

b) public boolean addAll(Collection c) :- It is used to insert the specified collection elements in the existing collection(For merging the Collection)

c) public boolean retainAll(Collection c) :- It is used to retain all the elements from existing element. (Common Element)

d) public boolean removeAll(Collection c) :- It is used to delete all the elements from the existing collection.

e) public boolean remove(Object element) :- It is used to delete an element from the collection based on the object.

f) public int size() :- It is used to find out the size of the Collection [Total number of elements available]

g) public void clear() :- It is used to clear all the elements at once from the Collection.

h) default boolean removeIf(Predicate<T> p) : It will remove the elements from the Collection based on boolean expression using Predicate.

i) public boolean contains(Object obj) : Used to search an Object in the given collection.

j) public Object[] toArray() : Used to convert the given Collection into array object.

All the above methods of Collection interface will be applicable to all the sub interfaces like List, Set and Queue.